Introduction

This set of notes explains how modern distributed storage systems evolved and why Hadoop Distributed File System (HDFS) was created. It builds from the historical growth of data, limitations of traditional systems, and the need for distributed architectures. It covers distributed system fundamentals, scaling strategies, file system evolution, failure handling, and the CAP theorem, all leading to the design philosophy of HDFS.

Evolution of Data Processing Systems

Three Eras of Data Engineering

Data processing evolved in response to increasing **volume, variety, and velocity** of data.

Era	Time Period	Focus	Key Constraint	Core Solution	Typical Role
Pre-Hadoop	1980s–Early 2000s	Data Warehousing	Volume & Cost	Vertical Scaling	DBA / ETL Developer
Hadoop Era	~2003–Early 2010s	Big Data	Scale & Variety	Horizontal Scaling	Big Data Engineer
Modern Cloud	Mid-2010s–Today	Flexible Analytics	Velocity & Flexibility	Decoupled Compute & Storage	Modern Data Engineer

Pre-Hadoop Era

- Focused on **structured data** and relational databases
- Used **MPP data warehouses** (Teradata, Netezza)
- Shared-nothing architecture
- Scaling required expensive proprietary hardware

Limitations

- Extremely costly to scale
- Could not handle unstructured data
- Hit performance and storage limits

Hadoop Era

Driven by internet-scale companies facing explosive data growth.

Key Triggers

- Google File System (2003)
- MapReduce (2004)
- Open-source Hadoop (2006)

Key Innovations

- Commodity hardware instead of specialized machines
- Software-managed fault tolerance
- Horizontal scaling

Core Components

- HDFS → distributed storage
- MapReduce → distributed processing

Impact

- Reduced storage cost dramatically
- Enabled petabyte-scale data lakes
- Proved horizontal scaling is superior economically

Post-Hadoop / Modern Era

MapReduce became too slow for real-time and iterative workloads.

New Needs

- Faster processing
- Interactive analytics
- Flexible pipelines

Modern Solutions

- Apache Spark (in-memory computing)
- Cloud-native storage
- Data Lakehouse architecture

Fundamentals of Distributed Systems

Definition

A distributed system is a collection of independent computers (nodes) that coordinate over a network to appear as a single system.

Centralized vs Distributed

Aspect	Centralized	Distributed
Control	Single machine	Multiple nodes
Failure Impact	Total outage	Partial survivability
Scalability	Limited	High

Complexity	Low	High

Advantages

Scalability

- Capacity increases by adding nodes, allowing systems to handle growing workloads smoothly.

Fault Tolerance

- Failure of one node does not bring down the entire system.

High Availability

- Replication ensures services remain accessible even during failures.

Geographic Distribution

- Placing nodes near users reduces latency and improves performance.

Challenges

Complexity

- Coordinating many machines requires advanced algorithms and protocols.

Network Dependency

- Performance depends on network reliability and speed.

Consistency Management

- Keeping replicated data synchronized is difficult at scale.

Debugging Difficulty

- Failures are harder to trace across many machines.

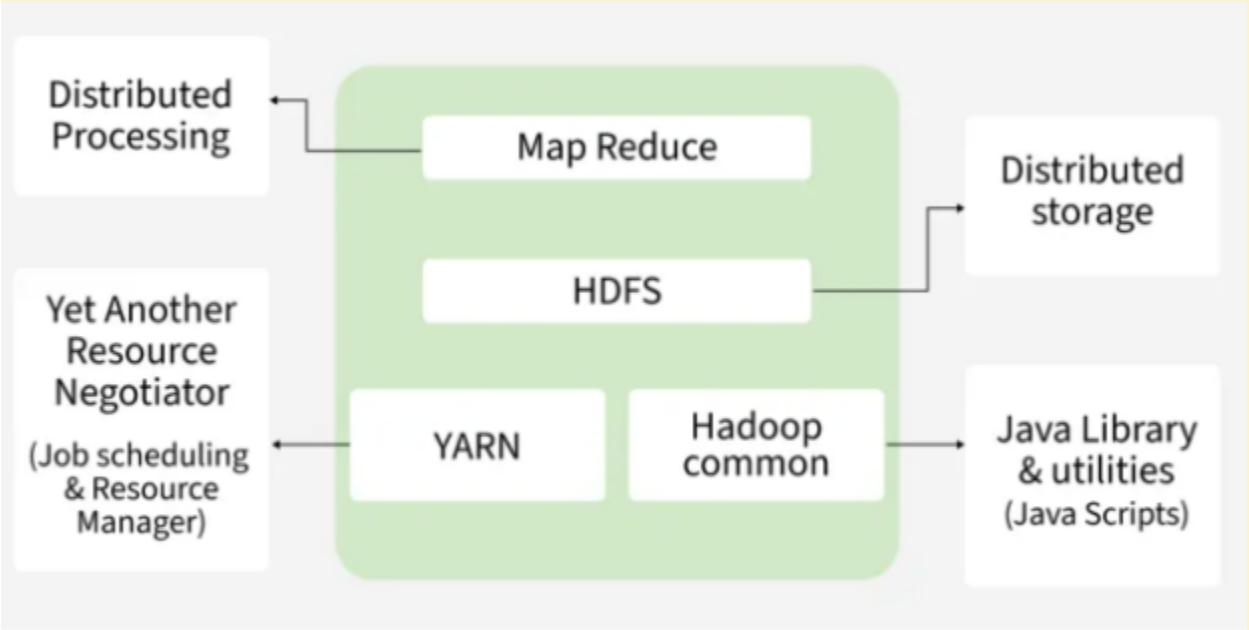
Hadoop Ecosystem

Definition

Hadoop is an open-source Java framework for distributed storage and processing using commodity hardware.

Core Components

- HDFS → storage layer
- MapReduce → processing layer
- YARN → resource management
- Hadoop Common → utilities



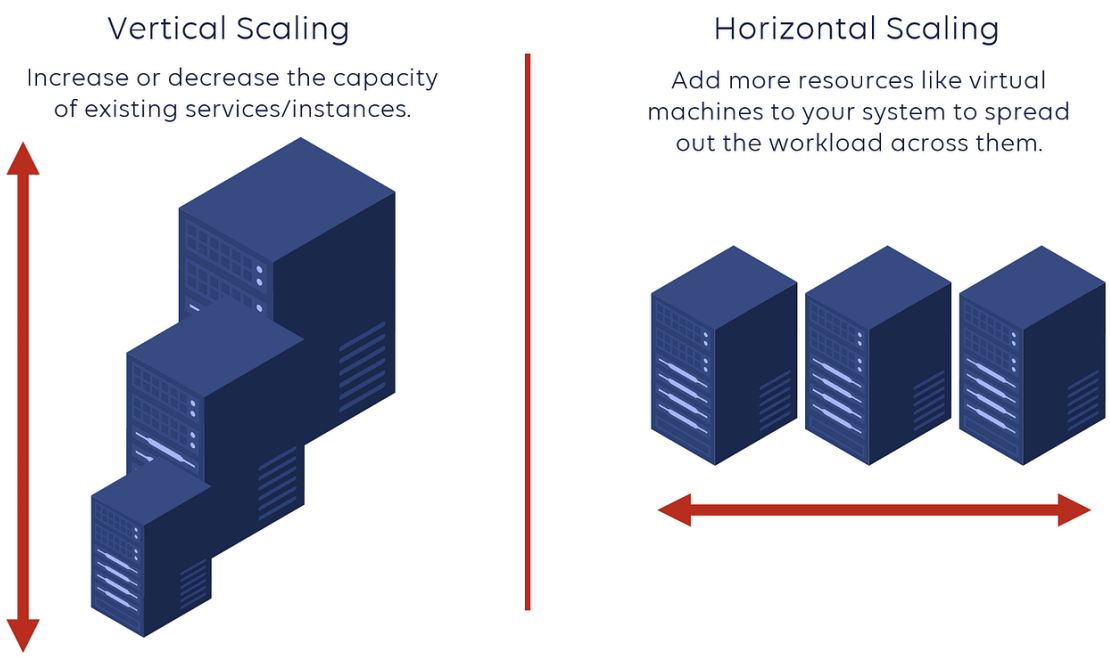
Hadoop architecture

Evolution of File Systems

As data kept growing beyond the limits of single machines, organizations realized that simply upgrading hardware was not sustainable. This need for scalable storage led to new approaches in file system design.

Vertical vs Horizontal Scaling

Factor	Vertical Scaling	Horizontal Scaling
Method	Add resources to one machine	Add more machines
Cost	Exponential	Linear
Limit	Physical hardware limit	Practically unlimited
Failure Risk	Single point	Distributed risk
Complexity	Low	High



Economic Insight

Commodity hardware became ~10× cheaper per TB than enterprise systems, driving distributed storage adoption.

Network File Systems

NFS (Network File System)

- Enabled remote file access
- Still centralized
- Single point of failure
- No replication

AFS (Andrew File System)

Key innovation: **Location Transparency**

Users access files without knowing the server location.

Benefits:

- Easier scaling
- Replica redirection
- Transparent migration

Blocks and Metadata Separation

Traditional File System

- Metadata and data stored together
- Suitable for single-machine systems

Solution to Traditional File System : HDFS Model

Component	Role
NameNode	Stores metadata
DataNodes	Store actual data blocks

Why Separation Works

- Metadata is small and fits in RAM
- Data is massive and must be distributed
- Central metadata enables coordination

HDFS Design Philosophy

Move Computation to Data

Sending small code to large data is faster than moving large data across the network.

Core Principles

- Hardware failures are expected
- Optimized for streaming reads
- Designed for large datasets

- Write-once-read-many model

Handling Failures

Fail-Stop Failure

Node completely stops.

Handled via replication.

Network Partition

Nodes cannot communicate but still operate.

Indistinguishable from crash failure.

Byzantine Failure

Node sends incorrect or malicious data.

HDFS assumes non-malicious nodes and uses checksums.

CAP Theorem

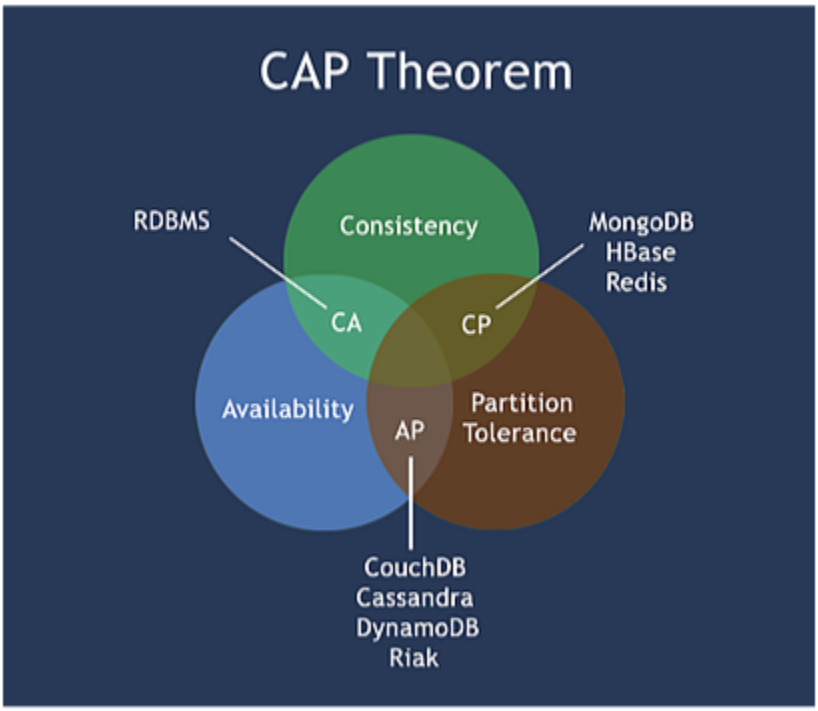
Definition

A distributed system cannot guarantee all three during a partition:

- Consistency
- Availability
- Partition tolerance

Since partitions are inevitable, systems choose between:

- CP
- AP



CP Systems

Prioritize correctness over availability.

Examples:

- Banking systems
- HDFS NameNode

AP Systems

Prioritize availability with eventual consistency.

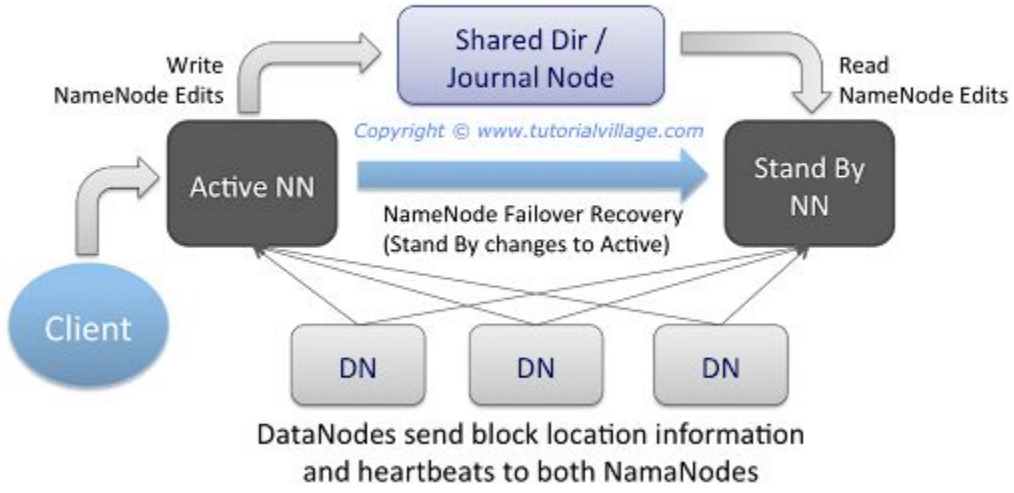
Examples:

- Social media feeds
- DynamoDB

CAP in HDFS

NameNode → CP

- Metadata inconsistency is catastrophic
- Uses active-standby model
- Temporary unavailability during failover



DataNodes → AP

- Block reads remain available

- Extra replicas tolerated
- Eventual consistency achieved

Conclusion

Key Insights

- Distributed systems trade simplicity for scalability and resilience
- Horizontal scaling enables cost-effective growth
- Separating metadata from data is fundamental to HDFS
- Failures are normal and must be designed for
- CAP theorem forces trade-offs
- HDFS smartly combines CP and AP strategies
- Moving computation to data is essential for performance

Key Terms

- **Distributed System** : Multiple independent computers acting as one system
- **Horizontal Scaling** : Adding more machines to increase capacity
- **Vertical Scaling** : Adding more power to one machine
- **HDFS** : Hadoop’s distributed storage system
- **NameNode** : Stores file system metadata
- **DataNode** : Stores actual data blocks
- **Replication** : Storing multiple copies of data

- **Fail-Stop Failure** : Node stops completely
- **Network Partition** : Communication breakdown between nodes
- **Byzantine Failure** : Malicious or inconsistent behavior
- **CAP Theorem** : Impossibility of guaranteeing C, A, and P simultaneously
- **CP System** : Consistency prioritized over availability
- **AP System** : Availability prioritized with eventual consistency
- **Location Transparency** : Accessing data without knowing physical location
- **Write-once-read-many** : File update model in HDFS